

Universidad Católica Boliviana "San Pablo"

Facultad de Ingeniería

Carrera de Ingeniería de Sistemas

Segunda Evaluación de Tecnologías Web II 1-2025 – Primera Opción

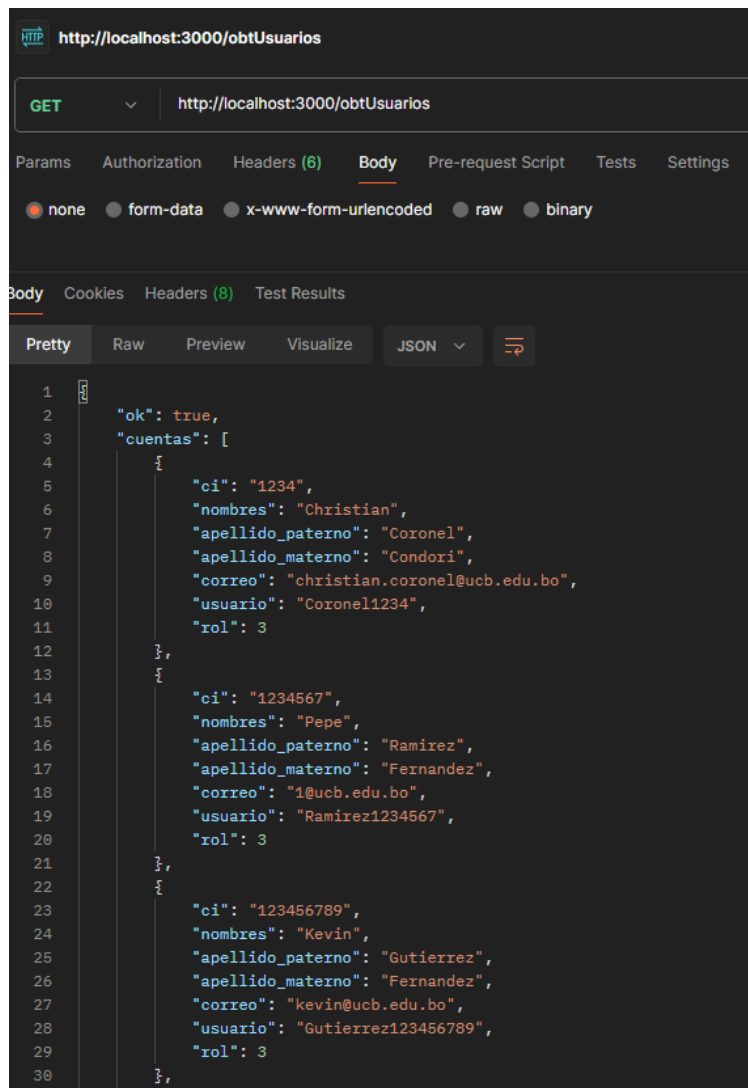
Nombre: Alan Franz Flores Campos

1. Análisis del Proyecto Integrador (GESTION DE TALLER DE GRADO)

❖ Identificación clara del endpoint

Para la implementación de este microservicio utilice dos endpoints, uno de GET y uno de POST

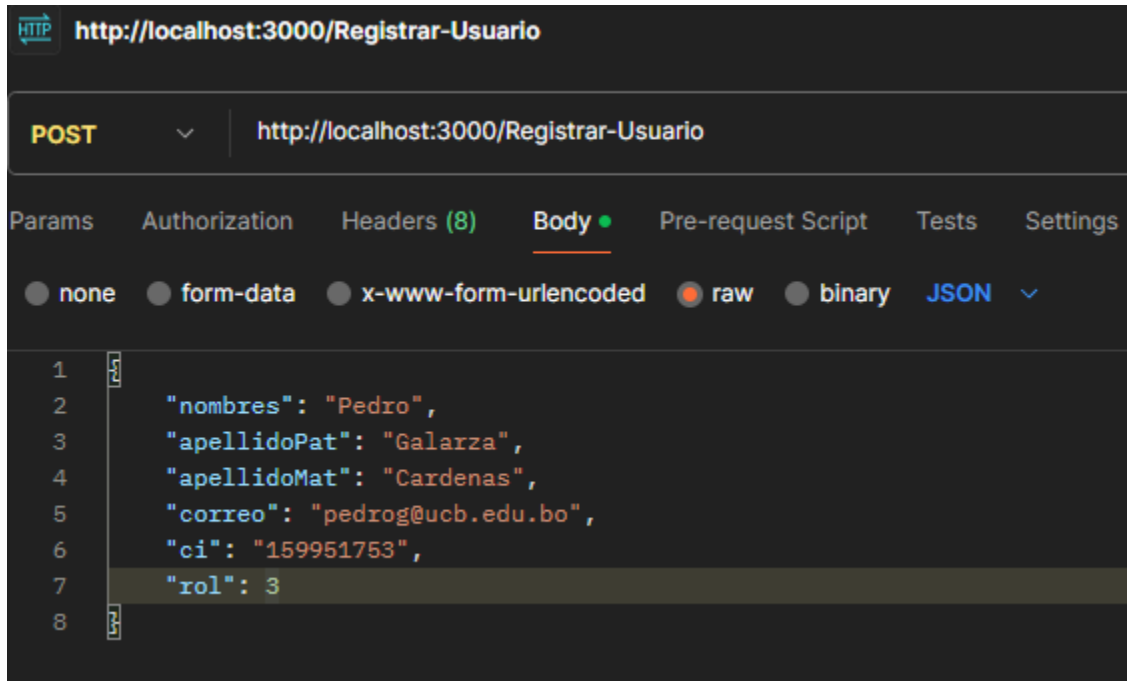
-GET de usuarios registrados en el portal web:



The screenshot shows a REST client interface with a GET request to `http://localhost:3000/obtUsuarios`. The response is displayed in JSON format, showing a successful status (`ok: true`) and a list of three user accounts. Each account contains fields for `ci`, `nombres`, `apellido_paterno`, `apellido_materno`, `correo`, `usuario`, and `rol`.

```
1  {
2    "ok": true,
3    "cuentas": [
4      {
5        "ci": "1234",
6        "nombres": "Christian",
7        "apellido_paterno": "Coronel",
8        "apellido_materno": "Condori",
9        "correo": "christian.coronel@ucb.edu.bo",
10       "usuario": "Coronel1234",
11       "rol": 3
12     },
13     {
14       "ci": "1234567",
15       "nombres": "Pepe",
16       "apellido_paterno": "Ramirez",
17       "apellido_materno": "Fernandez",
18       "correo": "1@ucb.edu.bo",
19       "usuario": "Ramirez1234567",
20       "rol": 3
21     },
22     {
23       "ci": "123456789",
24       "nombres": "Kevin",
25       "apellido_paterno": "Gutierrez",
26       "apellido_materno": "Fernandez",
27       "correo": "kevin@ucb.edu.bo",
28       "usuario": "Gutierrez123456789",
29       "rol": 3
30     }
31   ]
32 }
```

-POST de registro de usuarios en el portal web:



❖ Justificación de la elección del endpoint

Se eligió un GET para obtener la lista de usuarios ya que es una operación de lectura que no modifica el estado del sistema, cumpliendo con los principios REST.

El POST fue elegido para registrar un nuevo usuario, ya que se trata de una operación de creación, y este método es el más adecuado para enviar datos que serán almacenados en el servidor.

❖ Descripción técnica del endpoint (estructura, método, formato de datos)

-Método: GET

Formato de respuesta: JSON

Ejemplo de respuesta:



```

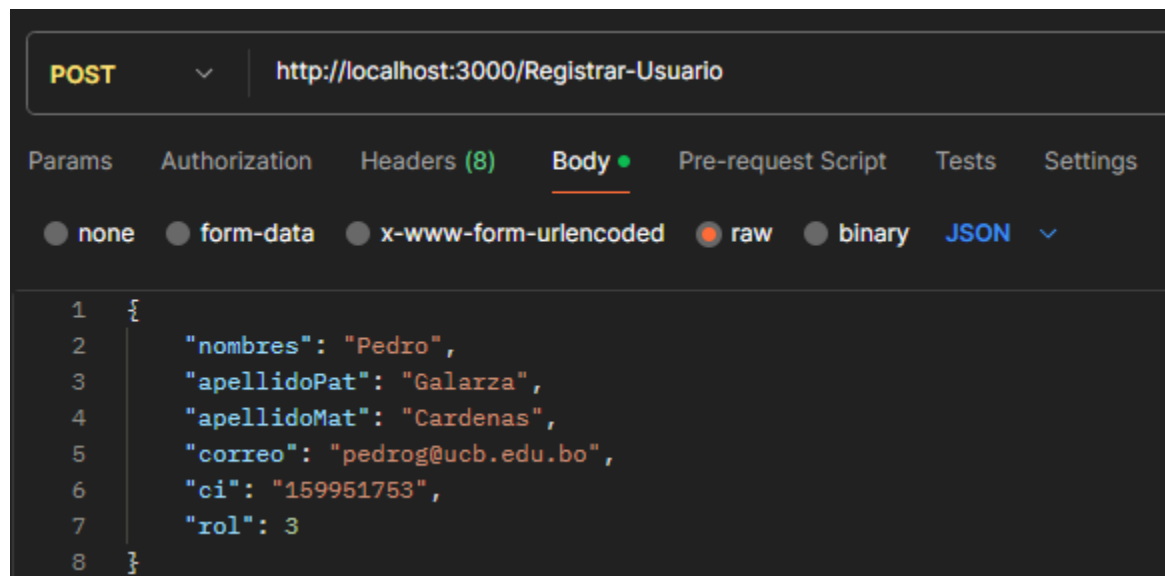
1  {
2    "ok": true,
3    "cuentas": [
4      {
5        "ci": "1234",
6        "nombres": "Christian",
7        "apellido_paterno": "Coronel",
8        "apellido_materno": "Condori",
9        "correo": "christian.coronel@ucb.edu.bo",
10       "usuario": "Coronel1234",
11       "rol": 3
12     },
13     {
14       "ci": "1234567",
15       "nombres": "Pepa",
16       "apellido_paterno": "Ramirez",
17       "apellido_materno": "Fernandez",
18       "correo": "1@ucb.edu.bo",
19       "usuario": "Ramirez1234567",
20       "rol": 3
21     },
22     {
23       "ci": "123456789",
24       "nombres": "Kevin",
25       "apellido_paterno": "Gutierrez",
26       "apellido_materno": "Fernandez",
27       "correo": "kevin@ucb.edu.bo",
28       "usuario": "Gutierrez123456789",
29       "rol": 3
30     }
31   ]
32 }

```

Método: POST

Formato de datos enviados: JSON

Ejemplo de body:



```

1  {
2    "nombres": "Pedro",
3    "apellidoPat": "Galarza",
4    "apellidoMat": "Cardenas",
5    "correo": "pedrog@ucb.edu.bo",
6    "ci": "159951753",
7    "rol": 3
8  }

```

2. Diseño del Microservicio

❖ *Objetivo del microservicio claramente definido*

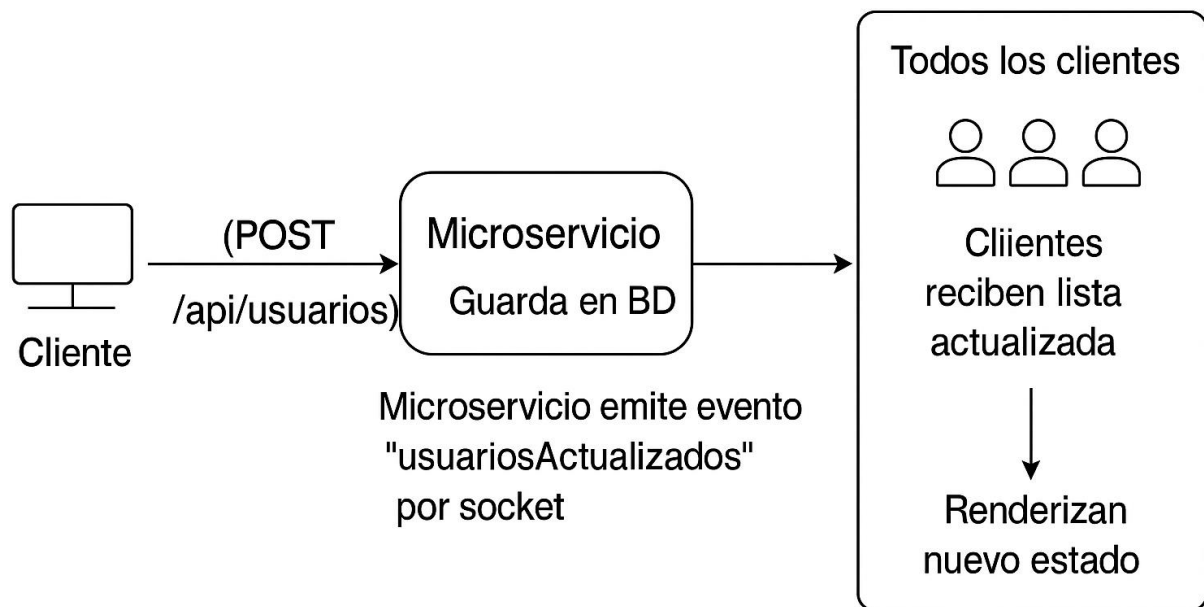
El objetivo del microservicio es permitir el registro de nuevos usuarios en el portal web del sistema de gestión de Taller de Grado y, al hacerlo, actualizar en tiempo real la lista de

usuarios disponibles en todos los clientes conectados, mejorando la sincronización de la información.

❖ *Elección y justificación de la tecnología de comunicación*

Para la comunicación en tiempo real, se utilizó Socket.IO, ya que permite una conexión persistente entre servidor y cliente. Esta tecnología es ideal para emitir eventos como la actualización de listas sin necesidad de recargar la página al momento de consumir los endpoints

❖ Diagrama del flujo de integración



3. Implementación Técnica

❖ *Desarrollo del microservicio funcional*

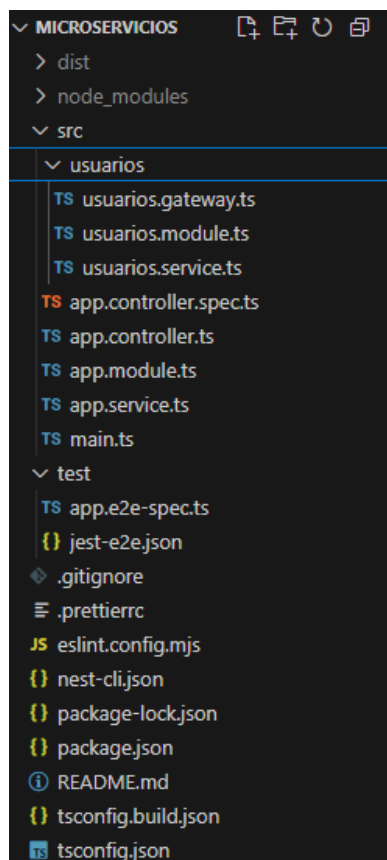
El microservicio fue desarrollado utilizando el framework NestJS basado en Express para los endpoints REST y Socket.IO para la comunicación en tiempo real. Se utilizó una base de datos en PostgreSQL para almacenar los datos de los usuarios, también se usó la dependencia axios

Para la comunicación en tiempo real se implementó Socket.IO mediante un `@WebSocketGateway()`, mientras que el manejo de lógica de negocio se delegó al servicio `UsuariosService`.

El microservicio permite registrar usuarios y emitir automáticamente la lista actualizada a todos los clientes conectados, garantizando así sincronización en tiempo real.

Internamente, al recibir un nuevo usuario mediante el evento `createUsuario`, el microservicio hace una solicitud HTTP (`axios.post`) hacia un backend externo que registra el usuario, y luego recupera la lista completa de usuarios (`axios.get`). Finalmente, emite esta lista actualizada mediante `this.server.emit('findAllUsuarios', response)` a todos los clientes conectados.

Estructura:



`usuarios.gateway.ts`:

```
TS main.ts    TS usuarios.gateway.ts X    TS usuarios.module.ts    TS usuarios.service.ts

src > usuarios > TS usuarios.gateway.ts > ...
1  import { WebSocketGateway, SubscribeMessage, MessageBody, WebSocketServer } from '@nestjs/websockets';
2  import { UsuariosService } from './usuarios.service';
3  import { Server } from 'socket.io';
4
5  @WebSocketGateway()
6  export class UsuariosGateway {
7    constructor(private readonly usuariosService: UsuariosService) {}
8    @WebSocketServer()
9    server : Server
10
11    @SubscribeMessage('createUsuario')
12    async create(@MessageBody() data) {
13      const response = await this.usuariosService.create(data);
14      this.server.emit('findAllUsuarios', response);
15    }
16
17    @SubscribeMessage('findAllUsuarios')
18    findAll() {
19      return this.usuariosService.findAll();
20    }
21
22    @SubscribeMessage('findOneUsuario')
23    findOne(@MessageBody() id: number) {
24      return this.usuariosService.findOne(id);
25    }
26
27    @SubscribeMessage('updateUsuario')
28    update(@MessageBody() updateUsuarioDto) {
29      return this.usuariosService.update(updateUsuarioDto.id, updateUsuarioDto);
30    }
31
32    @SubscribeMessage('removeUsuario')
33    remove(@MessageBody() id: number) {
34      return this.usuariosService.remove(id);
35    }
36  }
37
```

usuarios.module.ts:

```
TS main.ts    TS usuarios.gateway.ts    TS usuarios.module.ts X    TS usuarios.service.ts

src > usuarios > TS usuarios.module.ts > ...
1  import { Module } from '@nestjs/common';
2  import { UsuariosService } from './usuarios.service';
3  import { UsuariosGateway } from './usuarios.gateway';
4
5  @Module({
6    providers: [UsuariosGateway, UsuariosService],
7  })
8  export class UsuariosModule {}
9
```

usuarios.service.ts:

```
TS main.ts TS usuarios.gateway.ts TS usuarios.module.ts TS usuarios.service.ts X
src > usuarios > TS usuarios.service.ts > UsuariosService > create
1  import { Injectable } from '@nestjs/common';
2  import axios from 'axios';
3
4  @Injectable()
5  export class UsuariosService {
6      async create(data) {
7          const response = await axios.post('http://localhost:3000/Registrar-Usuario', data)
8          const usuarios = await axios.get('http://localhost:3000/obtUsuarios')
9          return usuarios.data;
10     }
11
12     findAll() {
13         return `This action returns all usuarios`;
14     }
15
16     findOne(id: number) {
17         return `This action returns a #${id} usuario`;
18     }
19
20     update(id: number, updateUsuarioDto) {
21         return `This action updates a #${id} usuario`;
22     }
23
24     remove(id: number) {
25         return `This action removes a #${id} usuario`;
26     }
27 }
28
```

❖ Consumo correcto del endpoint externo

El microservicio consume endpoints externos definidos en el backend de la página principal, específicamente:

POST <http://localhost:3000/Registrar-Usuario>: para registrar un nuevo usuario.

GET <http://localhost:3000/obtUsuarios>: para obtener la lista completa de usuarios.

Esto se realiza usando la biblioteca axios, y el flujo asegura que los datos se mantengan actualizados en tiempo real.

❖ Procesamiento y transformación de los datos

Al recibir el evento createUsuario, se valida el contenido del objeto recibido desde el cliente antes de enviarlo al endpoint de registro. Luego, se transforma la respuesta del backend en

una lista clara y estructurada de usuarios, lista para ser consumida por el frontend y renderizada automáticamente.

Esto garantiza una respuesta consistente y amigable para el cliente, sin exponer detalles técnicos del proceso.

- ❖ Exposición de endpoint propio/documentado

Este microservicio no expone endpoints HTTP REST directamente, pero define eventos WebSocket documentados que actúan como sus "endpoints" propios. Estos eventos están definidos y gestionados mediante `@SubscribeMessage()` en el archivo `usuarios.gateway.ts`:

-`'createUsuario'`: registra un nuevo usuario.

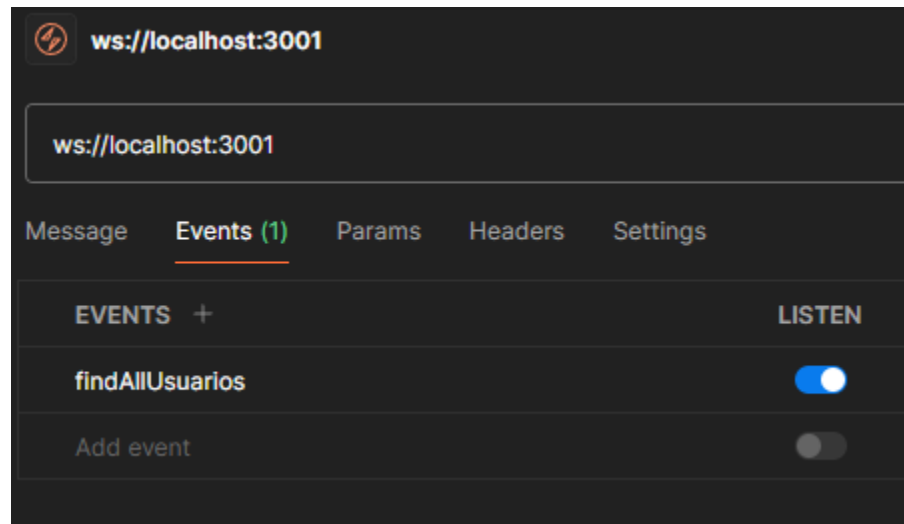
Cada uno de estos eventos acepta datos en formato JSON y devuelve respuestas también en formato JSON, siguiendo una estructura clara.

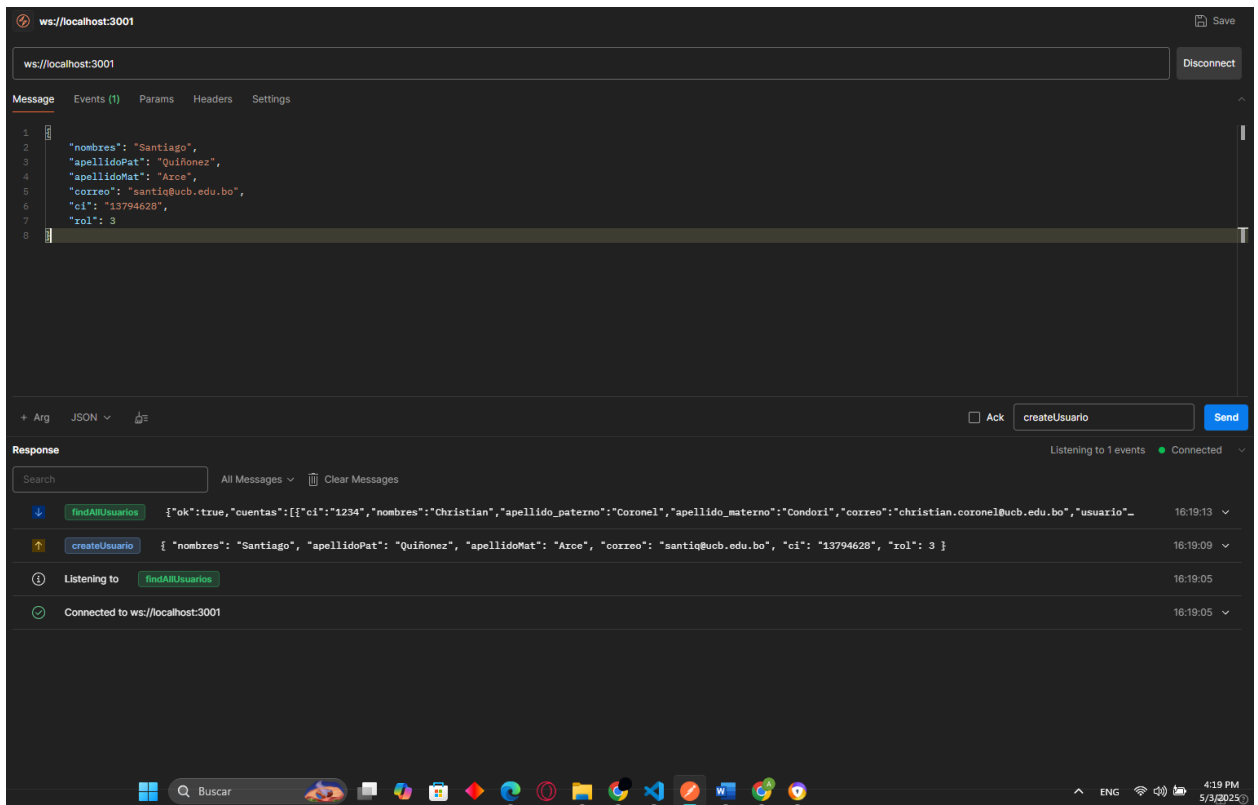
Toda esta funcionalidad está documentada en el archivo `README.md`

4. Pruebas y Documentación

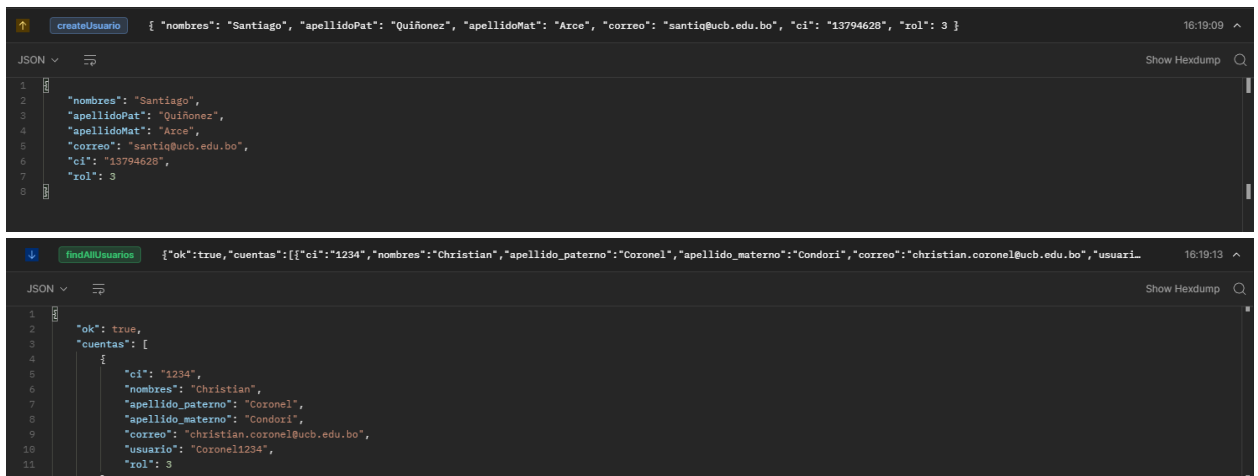
- ❖ Evidencias funcionales y pruebas unitarias (capturas, video, Postman, etc.)

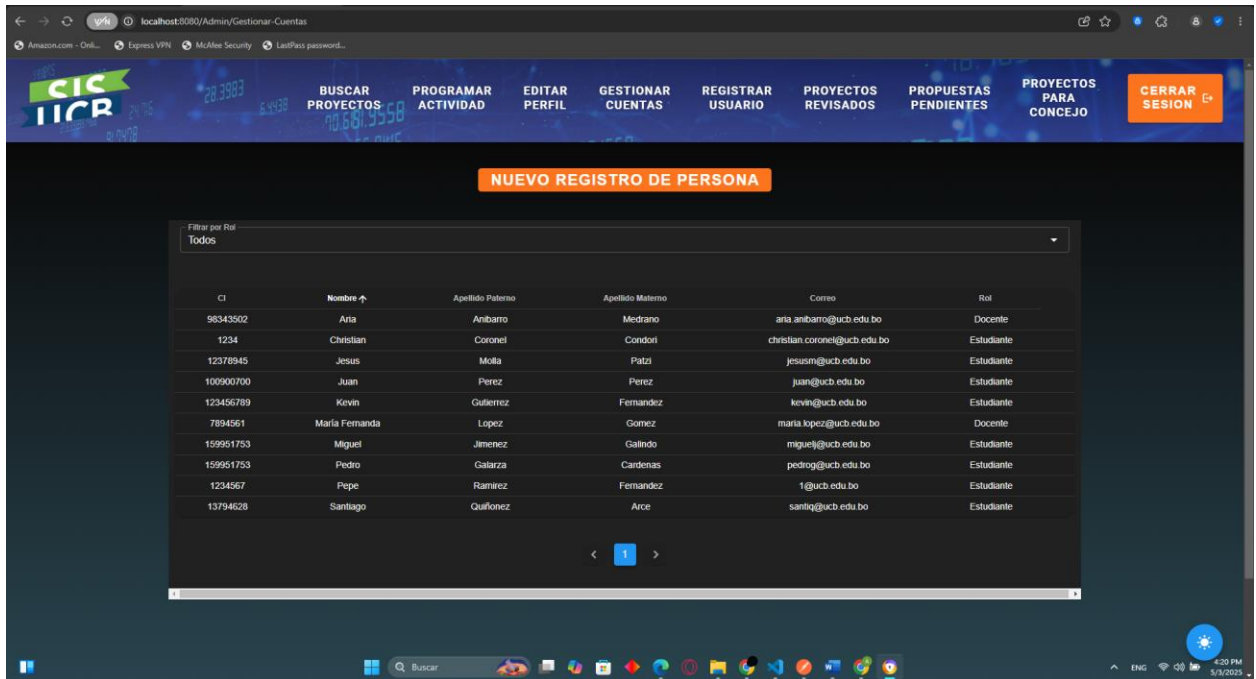
Se realizaron pruebas con Postman para verificar los endpoints.





Se evidencia el funcionamiento al momento “4:19”





Se ve que se actualiza en tiempo real el nuevo registro para el usuario

- ❖ Documentación clara del microservicio (README.md o PDF)



Microservicio de Usuarios

Este microservicio está desarrollado completamente con NestJS. Su propósito es gestionar usuarios a través de eventos WebSocket en tiempo real y consumir endpoints HTTP externos para registrar y obtener datos actualizados.

Estructura del Proyecto

```
src/
├── usuarios/
│   ├── usuarios.gateway.ts    # Gateway WebSocket con los eventos
│   ├── usuarios.module.ts     # Módulo de usuarios
│   └── usuarios.service.ts    # Lógica de negocio y consumo HTTP externo
├── app.controller.ts         # Controlador base NestJS (sin uso principal aquí)
├── app.module.ts             # Módulo raíz de la aplicación
└── main.ts                   # Punto de entrada principal
```

Instalación y Ejecución

Requisitos Previos

- Node.js v18+
- npm v9+
- Backend corriendo en `http://localhost:3000` con endpoints:
 - `POST /Registrar-Usuario`
 - `GET /obtenerUsuarios`

Pasos

```
npm install # Instalar dependencias
npm run start # Iniciar microservicio
```



Eventos WebSocket (Socket.IO)

Emitidos por el cliente

Evento	Acción	Ejemplo de payload
<code>createUsuario</code>	Crear usuario	<pre>{ nombres : Jesus , "apellidoPat": "Molla", "apellidoMat": "Patzi", "correo": "jesusm@ucb.edu.bo", "ci": "12378945", "rol": 3 }</pre>

Recibidos del servidor

Evento	Descripción	Ejemplo de respuesta
<code>findAllUsuarios</code>	Lista actualizada de usuarios	<pre>{ "ok": true, "cuentas": [{ "ci": "1234", "nombres": "Christian", "apellido_paterno": "Coronel", "apellido_materno": "Condori", "cor" ... }</pre>



Consumo de endpoints externos

```
POST /Registrar-Usuario # Registro de usuarios
GET /obtenerUsuarios    # Obtener lista completa
```

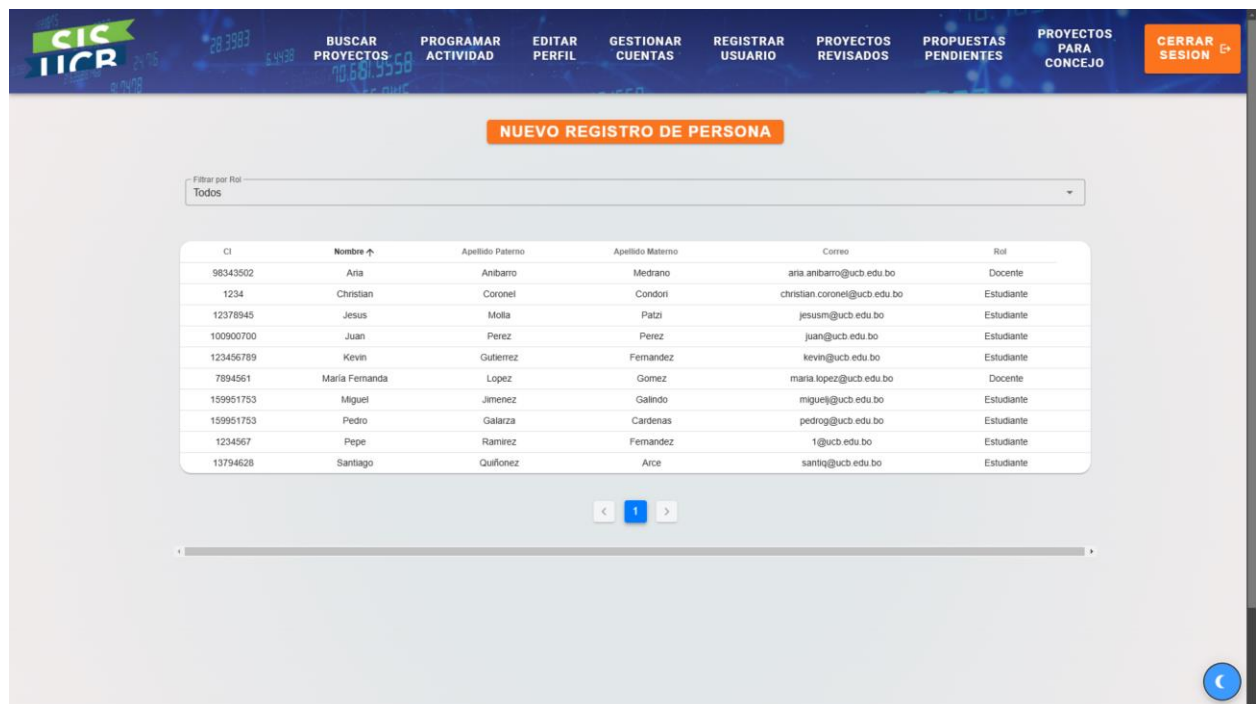
5. Presentación Final

- ❖ Explicación técnica del microservicio y su integración en PDF adjunto al repositorio

El documento técnico entregado junto al repositorio contiene una explicación detallada sobre la arquitectura, herramientas utilizadas y el flujo de datos dentro del microservicio de usuarios. Se describe cómo este servicio se comunica con el backend principal mediante llamadas HTTP, así como su integración en el sistema de Gestión de Taller de Grado mediante WebSocket para habilitar una experiencia de usuario en tiempo real. Además, se incluye un análisis del rol del microservicio en el ecosistema general del sistema, sus responsabilidades, y cómo puede escalarse o mantenerse en futuros desarrollos.

- ❖ Demostración funcional clara

Durante la presentación, se realiza una demostración práctica en la que se registra un nuevo usuario desde una interfaz cliente. Inmediatamente, todos los clientes conectados reciben de manera automática una lista actualizada de usuarios gracias al uso de eventos emitidos por WebSocket. Esta demostración permite visualizar de manera clara y comprensible cómo se garantiza la sincronización en tiempo real entre múltiples usuarios del sistema, lo que refleja la correcta implementación del patrón de arquitectura orientada a eventos y el cumplimiento de los requisitos funcionales del proyecto.



The screenshot displays a web application interface with a dark blue header containing navigation links: BUSCAR PROYECTOS, PROGRAMAR ACTIVIDAD, EDITAR PERFIL, GESTIONAR CUENTAS, REGISTRAR USUARIO, PROYECTOS REVISADOS, PROPUESTAS PENDIENTES, PROYECTOS PARA CONCEJO, and CERRAR SESION. Below the header, there is a section titled 'NUEVO REGISTRO DE PERSONA' with a filter dropdown set to 'Todos'. A table lists users with columns for CI, Nombre, Apellido Paterno, Apellido Materno, Correo, and Rol. The table contains 12 rows of user data. At the bottom, there is a pagination control showing '1' and a scrollbar.

CI	Nombre	Apellido Paterno	Apellido Materno	Correo	Rol
98343502	Aria	Anibarro	Medrano	aria.anibarro@ucb.edu.bo	Docente
1234	Christian	Coronel	Condori	christian.coronel@ucb.edu.bo	Estudiante
12378945	Jesus	Molla	Patzi	jesusm@ucb.edu.bo	Estudiante
100900700	Juan	Perez	Perez	juan@ucb.edu.bo	Estudiante
123456789	Kevin	Gutierrez	Fernandez	kevin@ucb.edu.bo	Estudiante
7894561	Maria Fernanda	Lopez	Gomez	maria.lopez@ucb.edu.bo	Docente
159951753	Miguel	Jimenez	Galindo	miguelj@ucb.edu.bo	Estudiante
159951753	Pedro	Galarza	Cardenas	pedrog@ucb.edu.bo	Estudiante
1234567	Pepe	Ramirez	Fernandez	1@ucb.edu.bo	Estudiante
13794628	Santiago	Quifonez	Arce	santiq@ucb.edu.bo	Estudiante